Credit Card Fraud Detection Using Machine Learning with Quantum-Inspired Restricted Boltzmann Machines

ARELLI PRAVEEN KUMAR
Roll No. 23K91A0513
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

BANOTHU AJAY
Roll No. 23K91A0529
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

D CHANDRIKA
Roll No. 23K91A0557
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

BAIRAGONI VIJAY
Roll No. 23K91A0525
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

KOTESHWAR RAO
[DESIGNATION] (project guide)
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

***Abstract*---**Fraud is rare among card transactions, so a classifier can score a high accuracy while finding almost no fraud, and fixed rules are quickly learned and bypassed. This paper builds and measures SecurePay AI, a fraud-screening application that follows a recent quantum-assisted study by training a Gaussian-Bernoulli restricted Boltzmann machine classifier three ways: persistent contrastive divergence, simulated-annealing sampling and a quantum-inspired sampler simulated on a CPU. A Flask service and a React interface offer single checks, CSV screening, analytics and retraining. On a synthetic, balanced set of 39,000 transactions the three-model ensemble reaches F1 94.5% (ROC-AUC 0.986), but the classical model flags nearly every transaction at the default threshold, and logistic regression, random forest and XGBoost score higher on the same split. Across five seeds the classical trainer is unstable while simulated annealing is steady. At 1 % fraud, undersampling cuts the precision of the baselines to about 0.2. A code audit found that the batch route skips feature scaling, which lowers accuracy from 94.8% to 50.2% on a fresh batch. The data are synthetic, so the figures are not comparable with the base paper's.

***Index Terms*---**class imbalance, credit card fraud, fraud detection, quantum annealing, restricted Boltzmann machine, simulated annealing, undersampling.

I. INTRODUCTION

Fraudulent card transactions are a tiny share of all payments: the dataset behind the base paper holds about 19 thousand fraud cases among 145 million transactions [1]. Banks have long used fixed rules, which analysts must keep rewriting and which fraudsters learn to avoid, and they now add learned classifiers. Two costs pull in opposite directions: a missed fraud is a direct loss, while a false alarm blocks a genuine customer and occupies an analyst. Accuracy hides both, because a model that approves everything is almost always right; an ANN and XGBoost, for example, both score about 99.96 % accuracy on imbalanced card data and differ only in fraud-class F1 [2].

The base paper [1] trains a Gaussian-Bernoulli restricted Boltzmann machine classifier (GRBMC) on a 39-thousand-transaction balanced subset and compares classical training with simulated-annealing and real quantum-annealer sampling. It uses proprietary data, reports no baseline from the usual classifier families, takes hours to train and offers no way to score a live transaction. Few readers can reproduce it, so it is unclear how much of its behaviour survives on ordinary hardware and how it compares with logistic regression, forests and boosting.

This work closes that gap at student scale. The contributions are: (1) an open NumPy implementation of the GRBMC with three trainers, wrapped in a Flask service and a React interface that scores one transaction or a CSV file; (2) a measured comparison with logistic regression, random forest and XGBoost, including five-seed stability and a threshold sweep; (3) a study of SMOTE, class weights and undersampling when fraud falls from 50 % to 1 % of the stream; and (4) an audit of the serving path with the measured cost of the defect found. Section II surveys related work, Section III the method, Section IV the results and Section V concludes.

A. System Hardware

All measurements used one desktop PC: Windows 11 Pro, an Intel Core i5-12500 processor and 15.7 GB of RAM, with no graphics processor. Every model trains on the CPU in NumPy, and the quantum samplers are classical simulations.

B. Software Requirements

Python 3.11.9 with NumPy 1.26.4, pandas 2.2.1, scikit-learn 1.5.0 [3], imbalanced-learn 0.12.4 [4], XGBoost 2.0.3 [5] and Matplotlib 3.9.0; Flask 3.1.0 with Flask-CORS for the API; React 19 and Vite 8 with Recharts for the browser interface.

II. LITERATURE SURVEY

Base paper. Neto et al. [1] work with 145 million anonymised card transactions from a Brazilian fintech, about 19 thousand of them fraud. Categorical fields are one-hot encoded, numeric fields z-scored, highly correlated columns removed and the remainder ranked with CatBoost. SMOTE, Tomek links and their combinations failed because the minority class was so small, so the authors undersampled by

hand to 39 thousand transactions, split 50/50 between classes and 80/20 into training and test sets. A GRBMC, in which the label units extend the visible layer, was trained for 50 epochs three ways: classical persistent contrastive divergence (200 hidden units, batch 512), simulated-annealing-assisted training on a D-Wave simulator and quantum sampling on D-Wave hardware (both 65 hidden units, batch 32). At the best epoch they report accuracy, precision, recall and F1 of 84, 86, 81 and 83 % (PCD, 2 minutes), 85, 94, 82 and 88 % (SAS, 4 hours) and 85, 91, 79 and 84 % (QS, 1.8 hours). The authors say that it remains open whether the advantage is general or specific to their data.

Classifiers and ensembles. Early comparisons found nonlinear ensembles stronger than linear models on card data [6]. A tuned random forest [7] trained on SMOTE-balanced data reached about 98 % accuracy and F1 on 94,682 transactions [8], and gradient boosting [5] is a standard strong baseline. On the harder IEEE-CIS benchmark (3.5 % fraud) the best fusion of random forest, XGBoost and LightGBM gives AUC 0.9405, AUPRC 0.6699 and F1 0.6373, and synthetic CTGAN fraud samples made results worse [9].

Imbalance and evaluation. SMOTE interpolates new minority points [10]; undersampling distorts posterior probabilities unless they are recalibrated [11]; and precision-recall curves are more informative than ROC curves when positives are rare [12]. Realistic fraud work also has to cope with delayed labels and changing patterns [13], and supervised classifiers gain from unsupervised anomaly scores [14].

Boltzmann machines. An RBM is trained by contrastive divergence [15], in its persistent form [16], and can classify directly when the label is part of the visible layer [17]; Gaussian visible units handle continuous inputs [18]. Simulated annealing [19] supplies the cooling schedule used to replace Gibbs sampling in the annealing-assisted variants.

TABLE I
REPRESENTATIVE RELATED WORK

Work	Approach	Data	Reported result	Limitation
[1]	GRBMC; PCD, SA, quantum sampling	145 M real, 39 k balanced	F1 83 / 88 / 84 %	Private data; hours; no UI
[8]	Random forest + SMOTE	94,682 online txns	Acc., F1 about 98 %	Resampled, accuracy-led
[9]	Fusion of 7 classifiers	IEEE-CIS, 590,540	AUPRC 0.670, F1 0.637	Small gain; CTGAN hurt
[2]	ANN vs XGBoost	Imbalanced card data	Both about 99.96 % acc.	Accuracy uninformative
[6]	LR, SVM, forest	Real card data	Ensembles stronger	Old; no deployment
[14]	Unsupervised + supervised	Large real stream	Scores complement	Not one compact model

Reading the base paper critically. Its strengths are the scale and realism of the data, access to annealing hardware and a systematic search over hidden units and learning-rate schedules. Its limits are equally clear from the text: results are given for one proprietary dataset and four metrics at the best of 50 epochs, chosen on the test split; metrics for several quantum-annealing epochs were unavailable because access to the hardware was limited; simulated annealing used only a few sweeps and oscillated strongly; the 4-hour simulated-annealing time is attributed partly to Julia-to-Python overhead on Windows; and no non-RBM classifier was trained for comparison. The authors themselves leave open whether the advantage is general. These are the points this project sets out to test on open, reproducible data.

Gap. The work surveyed studies classifiers and sampling methods separately, scores accuracy more than fraud-class recall, and stops at a notebook. No source tests whether the base paper's quantum-assisted GRBMC holds up against standard baselines on reproducible data, how it behaves at realistic imbalance, or whether it can sit behind a live scoring interface.

III. METHODOLOGY

Architecture. Fig. 1 shows four tiers. A React single-page application calls a Flask service through a Vite proxy; the service imports plain Python modules for preprocessing, the GRBMC, the samplers, training and evaluation; and all state is kept in local files (a CSV, a pickled pipeline, three pickled models and their histories). There is no SQL database, no login and no external call.

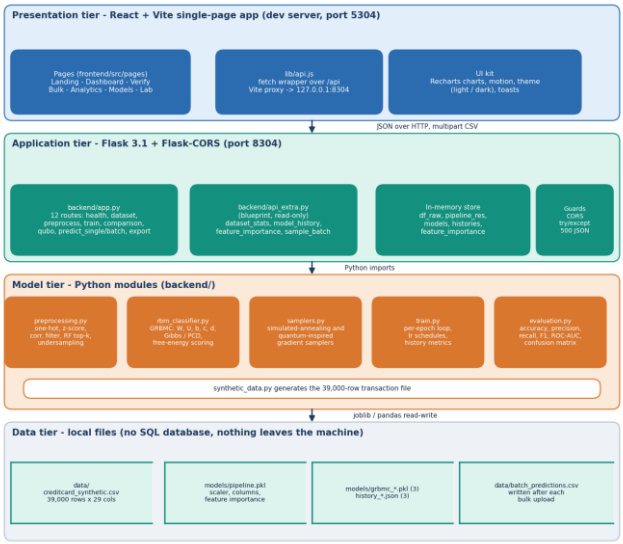


Fig. 1. SecurePay AI architecture: browser tier, Flask API, model modules and file storage.

Data. The file `creditcard_synthetic.csv` has 39,000 rows, half fraud, generated by the project (fixed seed). Fraud draws larger amounts (mean 368 against 85), more online and card-not-present use, more country mismatches and shifted anonymised features V3, V7, V10, V14 and V17 (Table II). It imitates the size and balance of the base paper's subset but is not real data. The backend builds its pipeline from an 8,000-row sample: one-hot encoding of merchant category and channel, z-scoring of 24 numeric columns, a correlation filter at 0.95 (which removes nothing), random-forest importance ranking (CatBoost is not installed, so this replaces it) keeping the top 18 features, and a stratified 80/20 split into 6,400 training and 1,600 test rows (805 fraud).

Balancing happens before the split, so selection saw the test rows.

TABLE II
DATASET STATISTICS BY CLASS (ALL 39,000 ROWS)

Property	Legitimate	Fraud
Rows	19,500	19,500
Mean amount	85.2	368.2
Median amount	80.3	352.1
Card present	80.5%	30.1%
Country matches card	95.0%	60.3%
Online channel	19.9%	60.3%
Mean of V3 / V17	0.00 / -0.02	1.51 / 1.31
Mean of V7 / V14	0.00 / -0.00	-1.20 / -0.79

Model. With unit variance the GRBMC energy of visible vector v (continuous), hidden vector h (binary) and label y (one-hot) is

$$E(v,h,y) = \tfrac{1}{2} \Sigma (v_i - c_i)^2 - \Sigma b_j h_j - \Sigma v_i W_{ij} h_j - \Sigma d_k y_k - \Sigma y_k U_{kj} h_j. \quad (1)$$

Because the visible bias term is the same for both labels, the class posterior used for prediction reduces to the free energy of the two label hypotheses,

$$p(y|v) \propto exp(d_y + \Sigma_j log(1 + exp(b_j + \Sigma_i v_i W_{ij} + U_{yj}))). \quad (2)$$

The update of every weight is the learning rate times the difference between the data correlation and the model correlation of the two units it connects, and the trainers differ only in how the model term is sampled: **PCD** runs one persistent Gibbs step with 200 hidden units and batch 512; **SAS** (Table III) anneals a population of 16 binary samples through 10 sweeps with inverse temperature rising from 0.5 to 10, computing the hidden field from the batch-mean visible vector; **QS** follows the same plan but replaces the temperature by a schedule A(s) = 0.8(1 − s), B(s) = s and flips each hidden unit at random with probability 0.3 A(s), imitating a transverse field. SAS and QS use 65 hidden units, batch 32 and a random 3,000-row subsample per epoch. These are simplified classical stand-ins for the base paper's path-integral simulator and quantum hardware, not reproductions. Learning rates decay as 0.01·exp(−0.08 t) for PCD and SAS and as 0.005·exp(−0.15 t) + 0.0005 for QS over 20 epochs. The ensemble averages the three fraud probabilities, labels fraud above 0.5 and assigns the risk bands LOW (below 0.4, approve), MEDIUM (0.4 to 0.7, challenge) and HIGH (above 0.7, decline and alert). Fig. 2 traces one scoring request.

TABLE III
SETTINGS OF THE THREE GRBMC TRAINERS

Trainer	Hidden / batch	Model-phase sampler	Learning rate	Rows per epoch
PCD	200 / 512	One persistent Gibbs step	0.01·exp(−0.08 t)	all 6,400
SAS	65 / 32	10 sweeps, 16 reads, β 0.5 to 10	0.01·exp(−0.08 t)	3,000 random
QS	65 / 32	10 sweeps, 16 reads, tunnelling flips	0.005·exp(−0.15 t) + 0.0005	3,000 random

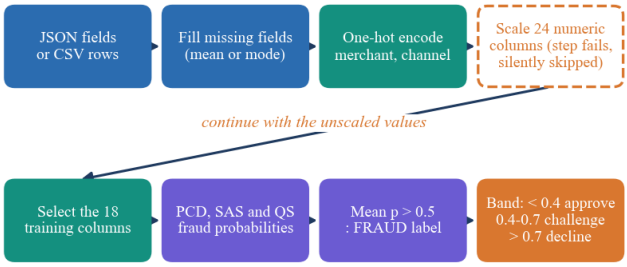


Fig. 2. Scoring flow of the API. The dashed scaler step raises an exception on the selected columns and is silently skipped.

Serving. Sixteen routes cover health, dataset and analytics, preprocessing, training, history, comparison, the QUBO matrix of each model, single and batch prediction and CSV export. Missing fields default to the dataset mean or mode, and any exception returns a JSON error with HTTP 500. The interface has a dashboard, a verify form, CSV bulk screening, four analytics tabs, a model-comparison page and a retraining lab.

Metrics. With TP, FP and FN the true positives, false positives and false negatives for the fraud class, precision is TP/(TP+FP), recall is TP/(TP+FN) and F1 is their harmonic mean. ROC-AUC measures ranking over all thresholds and is unaffected by the threshold problem of PCD; average precision (PR-AUC) is reported for the 1 % study because precision-recall curves are more informative for rare positives [12].

Evaluation protocol. Primary figures are final-epoch precision, recall, F1 and ROC-AUC at threshold 0.5 on the 1,600-row test split; the per-epoch best, which the interface also shows, is chosen on the test split and is therefore optimistic. Logistic regression, a 100-tree random forest and XGBoost (200 trees, depth 4) are trained on the same split. Stability uses five seeds; the imbalance study draws a 39,000-row set with 1 % fraud, splits it stratified into 31,200 and 7,800 rows (78 fraud in the test set), fits the scaler on training data only and resamples the training part only. A 20-check smoke test and an HTTP timing run exercise the running API. Every number in this paper is regenerated by `run_experiments.py`, `measure_latency.py` and `smoke_test.py` from fixed seeds, so a reader with the repository can repeat the study in under an hour on a CPU-only PC.

IV. RESULTS

TABLE IV
TEST-SPLIT RESULTS AT THRESHOLD 0.5 (%)

Model	Acc.	Prec.	Rec.	F1	AUC	F1, 5 seeds
PCD	58.8	54.9	100.0	70.9	0.986	69.2±23.1
SAS	94.5	96.7	92.2	94.4	0.986	94.4±0.1
QS	94.3	94.5	94.2	94.3	0.986	92.9±1.2
Ensemble	94.6	96.5	92.5	94.5	0.986	-
Log. reg.	96.7	96.3	97.1	96.7	0.995	-
Rand. forest	96.2	96.3	96.3	96.3	0.994	-
XGBoost	96.5	97.0	96.0	96.5	0.995	-

Test conditions. Shipped models and baselines are scored on the same 1,600 held-out rows (805 fraud) of the synthetic set; the five-seed column retrains each GRBMC for 20 epochs and reports the final epoch (mean ± sd). Training takes about 4 to 5 s per model on this PC, and 19 s for PCD on the full file.

Model comparison. SAS and QS reach F1 94.4% and 94.3%, and averaging all three gives 94.5% with ROC-AUC 0.986 (Table IV). The shipped PCD model has the same AUC (0.986) but at threshold 0.5 it labels 660 of 795 genuine transactions as fraud: its ranking is good and its probabilities are not calibrated, and moving the threshold changes the picture completely (Fig. 3a shows five retrainings, F1 69.2±23.1 for PCD against 94.4±0.1 for SAS). Logistic regression, random forest and XGBoost reach F1 96.7%, 96.3% and 96.5%; on the whole 39,000-row file they reach 96.9%, 96.9% and 97.1% against 94.5% for SAS. On this synthetic data the simple baselines are better, faster to fit (under a second) and stable.

Threshold. Sweeping the ensemble threshold shows how narrow the useful range is. At 0.4 precision is 0.73 and recall 1.00 (299 false positives); at 0.5 they are 0.97 and 0.93; at 0.6 precision is 1.00 but recall falls to 0.67; and at 0.7 nothing is flagged. No test transaction therefore reaches the HIGH band, so in practice the application can only approve or challenge on this data, and the 40 % and 70 % bands were not tuned to the model's output range.

What the models use. The random forest ranks Amount (0.36), V3 (0.13), Card Present (0.11), V17 (0.09), V7 (0.07), Country Match (0.06) highest, matching the way the data were generated: large amounts, card-not-present use, country mismatch and a few shifted anonymised features. Only 15 of the 20 saved SAS epochs and 8 of the 20 QS epochs exceed F1 0.9, so a model saved at an arbitrary epoch can be far from the best one; the five-seed figures use the last epoch for that reason. The saved histories report training times of 4.4 s (PCD), 4.2 s (SAS) and 3.4 s (QS), where the base paper reports minutes to hours; the difference comes from the 3,000-row subsampling and a heavily simplified sampler, not from faster quantum hardware.

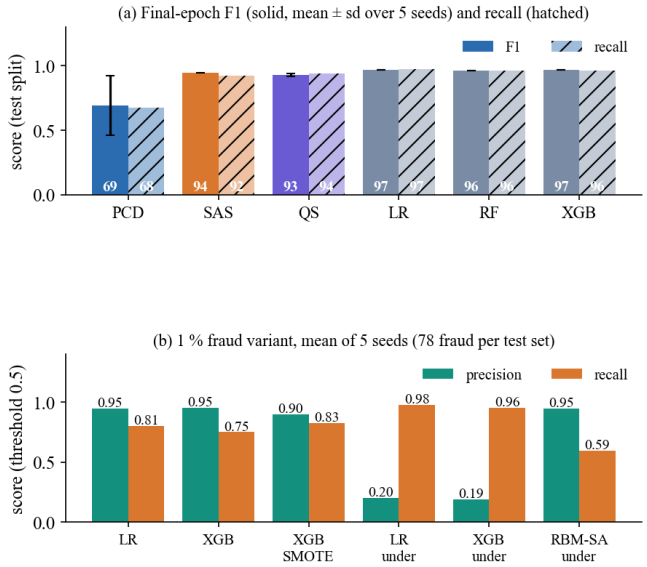


Fig. 3. Results: (a) F1 and recall of the trainers and baselines; (b) precision and recall when fraud is 1 % of the stream.

Imbalance. With 1 % fraud (Fig. 3b), unresampled logistic regression gives precision 0.95 and recall 0.81, XGBoost 0.95 and 0.75. SMOTE lifts XGBoost recall to 0.83 at precision 0.90, and class weights give 0.80 and 0.82. Random undersampling, the base paper's choice, raises recall above 0.95 for all three baselines but drops precision to 0.19 to 0.22, which means about 266 to 314 false alarms per 7,800 transactions. The GRBMC trained on the undersampled set keeps precision 0.95 but recall only 0.59 (ROC-AUC 0.93), and a three-signal rule scores F1 0.48.

TABLE V
RESULTS AT 1 % FRAUD: STRATEGY AND MODEL (MEAN OF 5 SEEDS)

Model, strategy	Prec.	Rec.	F1	PR-AUC	FP	FN
Log. reg., none	0.95	0.81	0.87	0.92	4	15
XGBoost, none	0.95	0.75	0.84	0.89	3	19
XGBoost, class weights	0.80	0.82	0.81	0.89	16	14
XGBoost, SMOTE	0.90	0.83	0.86	0.91	7	14
Log. reg., undersample	0.20	0.98	0.34	0.89	302	2
Rand. forest, undersample	0.22	0.97	0.36	0.88	266	2
XGBoost, undersample	0.19	0.96	0.32	0.86	314	3
GRBMC (SAS), undersample	0.95	0.59	0.72	0.73	3	32
Three-signal rule	0.56	0.42	0.48	-	27	45

Why the baselines win here. The generator shifts a handful of Gaussian features and a few categorical probabilities, so the classes are nearly linearly separable and logistic regression is close to optimal. A generative RBM spends part of its capacity modelling the inputs and is trained with a noisy gradient estimate, so it is plausible, though not tested here, that it needs real, harder data before any advantage appears. The gap to the baselines is about two F1 points on the 1,600-row split, larger than the 0.1 to 1.2 point seed spread of SAS and QS, so it is not noise.

Output screens. Fig. 4 is the verify page after the high-risk example of the user guide (amount 1,200, online, card not present, country mismatch): the models give 50.9 %

(PCD), 59.5 % (QS) and 94.7 % (SAS), the ensemble is 68.4 %, just inside the MEDIUM band, and the page recommends a 3-D Secure challenge. The three models disagree by more than forty points on one transaction, which is the instability of Section IV seen from the user's side. Fig. 5 is the bulk page after the bundled sample is uploaded: all 20 rows are flagged and the accuracy tile reads 50.0 %, the signature of the scaling defect rather than of the data, and the page itself warns that every row was flagged.

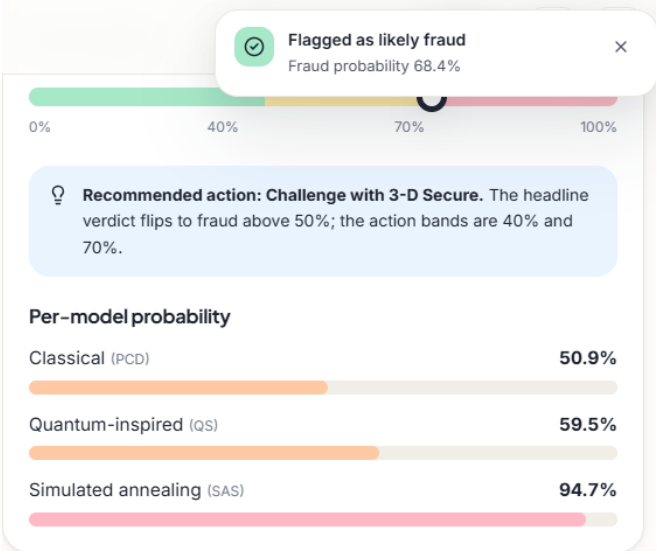


Fig. 4. Verify screen: a high-amount online transaction scored by the three models and the risk meter (the single-check route scales correctly).

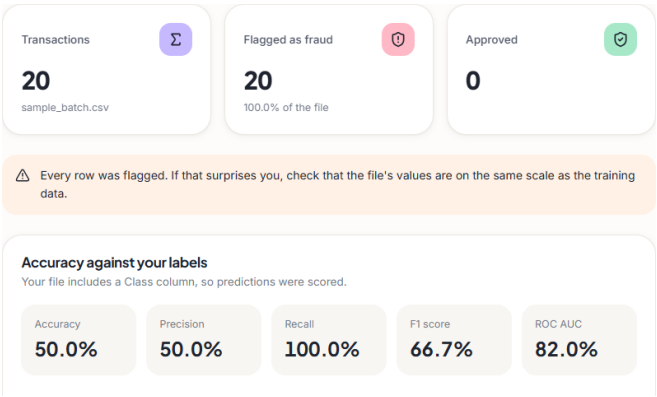


Fig. 5. Bulk screen: the bundled 20-row sample, all rows flagged because of the skipped scaling step.

Serving defect. The bulk route calls the scaler on the 13 selected numeric columns although it was fitted on 24, scikit-learn raises an error and a bare `except` hides it, so models receive unscaled values. On a fresh 2,000-row batch the ensemble then reaches accuracy 50.2% with 995 false positives, against 94.8% and 26 when scaling is applied offline. The bundled sample file (10 fraud rows) is flagged 20 of 20 as served and 11 when scaled (Fig. 5). The defect was recorded, not fixed.

TABLE VI
API LATENCY ON 127.0.0.1 (MS)

Request	n	Median	95th pct.
Single check, ensemble	100	16.9	32.5
Single check, PCD only	50	17.7	32.5
Single check, SAS only	50	17.1	32.9
20-row CSV batch	20	29.9	39.6
Dataset statistics	10	50.1	65.8
Health	50	15.4	16.6

Speed. A single check takes a median 17 ms (95th percentile 32 ms) and a 20-row CSV 30 ms on the Flask development server (Table VI), after five warm-up calls. These timings say nothing about concurrent load.

Comparison with the base paper. The base paper's Table 1 gives F1 of 83, 88 and 84 % for PCD, SAS and QS on real data, and 2 minutes, 4 hours and 1.8 hours of training. This project measures 71%, 94% and 94% in seconds on synthetic data whose classes are easy to separate (logistic regression reaches AUC 0.995). The datasets, sample sizes, simulators and epoch selection differ, so the two sets of figures are context for each other and neither supports a claim of improvement or of reproduction.

Failure cases. Four stand out. The shipped PCD model flags 660 of 795 genuine transactions. Early epochs of SAS and QS collapse to predicting one class (F1 of zero in 2 and 2 of 20 epochs of the saved runs), so the epoch at which training stops matters. The HIGH band is unreachable, as shown above. And on bulk uploads, which skip scaling, the ensemble flags almost everything. Where the data carry a strong signal the system works: the dataset analytics show that electronics, online and card-not-present transactions have the highest fraud rates, and the random-forest importances point the same way.

Practical reading. For an operations team the useful question is not accuracy but how many alerts a fixed analyst budget can absorb. Table V answers it for the 1 % stream: an unresampled linear model or boosted trees raise a handful of false alarms per 7,800 transactions, whereas undersampling raises hundreds, so the base paper's balancing choice, made necessary there by an extremely small minority class, is a costly default elsewhere unless probabilities are recalibrated afterwards [11]. A deployment would also choose its threshold on a validation set from the cost of a missed fraud against the cost of a review, which this prototype does not do.

Threats to validity. Internal: the threshold (0.5) and the feature count (18) are the product's defaults and were not tuned here, but balancing and feature ranking used all rows of the 8,000-row sample, so the test rows influenced selection. External: one synthetic generator, one random sample and one machine. Construct: the quantum-inspired samplers approximate nothing quantum, so no statement about quantum advantage follows. Statistical: five seeds give a rough spread, not a confidence interval, and the 1 % study has only 78 fraud rows per test set.

Limits. The test set is drawn from the same generator as the training data, the best-epoch curves in the interface are selected on test rows, and the imbalance study covers one

synthetic family, five seeds and a fixed threshold. Nothing here measures real fraud, drift, concurrency or user behaviour.

V. CONCLUSION

SecurePay AI delivers what the objectives asked for in the interface and the metrics: preprocessing, an imbalance study, F1, recall, precision and ROC-AUC reporting, and a screen that flags a transaction. The key finding is that the quantum-inspired RBM trainers reach F1 about 94% on synthetic data, while ordinary baselines reach 96% to 97%, so on this evidence the RBM is not the better model. PCD is unstable, undersampling is costly at realistic imbalance, and the largest practical risk was not the model but a swallowed exception in the serving path. Limits are the synthetic data, the classical simulation of quantum sampling, test-split epoch selection and the unfixed defect.

Future work: fix the scaling step and add a regression test; choose thresholds on a validation set and recalibrate PCD; train on real, time-ordered transactions with the base paper's data or a public set; implement path-integral annealing or use annealer access; and add authentication, monitoring for drift and a production server.

REFERENCES

[1] J. M. C. de Albuquerque Neto, G. C. do Amaral and G. P. Temporao, "Fraud detection in credit card transactions using quantum-assisted restricted Boltzmann machines," *IEEE Access*, vol. 14, pp. 62923-62932, 2026, doi: 10.1109/ACCESS.2026.3683308.

[2] J. Shi, "Credit card fraud detection based on ANN and XGBoost," in *Proc. 1st Int. Conf. E-commerce and Artificial Intelligence (ECAI)*, SciTePress, 2024, pp. 195-199, doi: 10.5220/0013212700004568.

[3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825-2830, 2011.

[4] G. Lemaitre, F. Nogueira and C. K. Aridas, "Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning," *J. Mach. Learn. Res.*, vol. 18, no. 17, pp. 1-5, 2017.

[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785-794.

[6] S. Bhattacharyya, S. Jha, K. Tharakunnel and J. C. Westland, "Data mining for credit card fraud: A comparative study," *Decision Support Systems*, vol. 50, no. 3, pp. 602-613, 2011.

[7] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.

[8] A. H. M. Aburbeian and H. I. Ashqar, "Credit card fraud detection using enhanced random forest classifier for imbalanced data," arXiv:2303.06514, Mar. 2023.

[9] X. Han and C. Wu, "Validation-stage combinatorial fusion analysis for imbalanced credit-card fraud detection," arXiv:2606.10393, Jun. 2026.

[10] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321-357, Jun. 2002.

[11] A. Dal Pozzolo, O. Caelen, R. A. Johnson and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in *Proc. IEEE Symp. Series on Computational Intelligence (SSCI)*, 2015, pp. 159-166.

[12] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, Art. no. e0118432, 2015.

[13] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi and G. Bontempi, "Credit card fraud detection: A realistic modeling and a novel learning strategy," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 8, pp. 3784-3797, Aug. 2018.

[14] F. Carcillo, Y.-A. Le Borgne, O. Caelen, Y. Kessaci, F. Oble and G. Bontempi, "Combining unsupervised and supervised learning in credit card fraud detection," *Information Sciences*, vol. 557, pp. 317-331, 2021.

[15] G. E. Hinton, "Training products of experts by minimizing contrastive divergence," *Neural Computation*, vol. 14, no. 8, pp. 1771-1800, Aug. 2002.

[16] T. Tieleman, "Training restricted Boltzmann machines using approximations to the likelihood gradient," in *Proc. 25th Int. Conf. Machine Learning (ICML)*, 2008, pp. 1064-1071.

[17] H. Larochelle, M. Mandel, R. Pascanu and Y. Bengio, "Learning algorithms for the classification restricted Boltzmann machine," *J. Mach. Learn. Res.*, vol. 13, pp. 643-669, 2012.

[18] R. Liao, S. Kornblith, M. Ren, D. J. Fleet and G. Hinton, "Gaussian-Bernoulli RBMs without tears," arXiv:2210.10318, 2022.

[19] S. Kirkpatrick, C. D. Gelatt Jr. and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671-680, 1983.